Corrigé - Examen écrit d'algorithmes et systèmes distribués

Master 1 informatique 2025/2026 - 1ère session, novembre 2025 - E. Fabiani

Exercice 1 - Détection de nœuds frères dans un arbre BFS (10 points)

Définition : deux nœuds sont frères si et seulement si ils sont connectés par un canal **et** ont le même père.

1.2 Compréhension (2 points)

Règle : si plusieurs *search* arrivent au même tour, le père retenu est celui du canal d'indice le plus élevé (la boucle `for i := range in` écrase `parent` à chaque *search* : le dernier indice gagne). Canaux utiles, au format *nœud:indice* : A0-B1, A1-E1, A2-D1, B2-C0, B0-F1, C1-D0, C2-J1, C3-G1, D2-E0, D3-H1, D4-K0, E2-H2, F0-I0, G0-I1, G2-J0, I2-J3, J2-K2, K1-H0.

Tour	Nœuds atteints	Search reçus (canal)	Père retenu
0	B, D, E	un seul chacun (venant de A)	B->A, D->A, E->A
1	C	B (canal 0), D (canal 1)	D (indice le plus élevé)
	F	B (canal 1)	B
	H	D (canal 1), E (canal 2)	E (indice le plus élevé)
	K	D (canal 0)	D
2	G	C (canal 1)	C
	I	F (canal 0)	F
	J	C (canal 1), K (canal 2)	K (indice le plus élevé)

Arbre obtenu : A a pour fils B, D, E ; B a pour fils F ; D a pour fils C, K ; E a pour fils H ; C a pour fils G ; F a pour fils I ; K a pour fils J.

Groupes de même père : {B, D, E}, {C, K}, et des singletons. Seule la paire {D, E} est reliée par un canal (D2-E0). B n'est relié ni à D ni à E ; C et K ne sont pas voisins.

Réponse : une seule fratrie, {D, E}.

Pièges : H et K sont voisins (K1-H0) et au même niveau, mais de pères différents (E et D) ; de même G, I, J sont deux à deux voisins (G-I, G-J, I-J) mais ont trois pères différents (C, F, K). Ce ne sont donc pas des frères.

1.3 Première version : détection après construction de l'arbre (4 points)

Idée : le BFS ne mémorise que l'*indice* du canal vers le père (`parent`). Il faut donc d'abord apprendre l'*identifiant* du père (phases 1-2), puis le diffuser à tous les voisins (phase 3) et comparer (phase 4). Chaque phase est exactement un appel à `communication`, pour rester synchrone. Code à ajouter après la boucle du BFS :

```
// Phase 1 : chaque noeud envoie son identifiant a ses fils (null ailleurs)
for i := range messOut {
    messOut[i] = null
}
for k := 0; k < nbfils; k++ {
    messOut[tabFils[k]] = id
}
communication(in, messIn, out, messOut)
```

```
// Phase 2 : memorisation de l'identifiant du pere
idPere := nonDef
if !racine {
    idPere = messIn[parent]   // parent = indice du canal qui mene au pere
}

// Phase 3 : envoi de l'identifiant du pere a TOUS les voisins (et reception)
for i := range messOut {
    messOut[i] = idPere
}
communication(in, messIn, out, messOut)

// Phase 4 : un voisin ayant le meme pere que moi est un frere
tabFrere := make([]int, len(in))
nbfrere := 0
for i := range messIn {
    if idPere != nonDef && messIn[i] == idPere {
        tabFrere[nbfrere] = i      // meme indice en entree et en sortie
        nbfrere++
    }
}
```

Remarques : la racine n'a pas de père (`idPere = nonDef`), elle ne détecte donc aucun frère, d'où la garde sur `racine` et `idPere != nonDef`. Un voisin qui n'envoie rien d'utile envoie `null` (-1) ou l'identifiant de son propre père, qui ne peut jamais être égal à `idPere` pour le père (son père est le grand-père) ni pour un fils (son père est moi).

1.4 Deuxième version : détection pendant la construction (4 points)

Idée : un nœud X reçoit son premier *search* au tour *t*. Ses frères (même père, donc même niveau) le reçoivent aussi au tour *t* et renvoient chacun un *search* au tour *t+1*. Au tour *t+1*, X reçoit donc leurs messages, dont le champ *parent* est le père commun. Si *parent* du message est égal à l'identifiant du père de X, l'émetteur est un frère. À ce tour, X ne reçoit de *search* que de nœuds de son niveau (le père et les autres pères potentiels envoient *null*, les fils ne sont pas encore atteints).

```
type message struct {
    nature int   // search ou null
    src    int   // identifiant de l'emetteur
    parent int   // identifiant du pere de l'emetteur (null pour la racine)
}

func node(id int, in []chan message, out []chan message, racine bool) {
    messIn := make([]message, len(in))
    messOut := make([]message, len(out))
    var visite bool = racine
    var parent int = nonDef     // indice du canal vers le pere
    var idPere int = null       // identifiant du pere (null pour la racine)
    var detecte bool = false    // vrai au tour qui suit la 1ere reception de search
    tabFrere := make([]int, len(in))
    nbfrere := 0

    for i := range out {
        if racine {
            messOut[i] = message{search, id, null}
        } else {
            messOut[i] = message{null, id, null}
        }
    }
    for nbTours := 0; nbTours < diametre+1; nbTours++ {
        communication(in, messIn, out, messOut)
        if !visite {
            for i := range in {
                if messIn[i].nature == search {
                    parent = i                // le dernier gagne (indice max)
                    idPere = messIn[i].src
                    visite = true
```

```
                    detecte = true
                }
            }
            if visite {
                for j := range out {
                    messOut[j] = message{search, id, idPere}
                }
            }
        } else {
            if detecte {   // tour qui suit la 1ere reception : freres
                for i := range in {
                    if messIn[i].nature == search && messIn[i].parent == idPere {
                        tabFrere[nbfrere] = i
                        nbfrere++
                    }
                }
                detecte = false
            }
            for i := range out {
                messOut[i] = message{null, id, null}
            }
        }
    }
    fmt.Println("id:", id, "parent:", parent, "freres:", tabFrere[:nbfrere])
}
```

Points à justifier : (1) la racine envoie *search* avec *parent* = null ; null (-1) n'est pas un identifiant valide, donc ce message ne peut jamais être pris pour celui d'un frère, et la racine, qui n'a jamais `detecte = true`, ne cherche pas de frères. (2) Un nœud à la distance d de la racine reçoit son premier *search* au tour d-1 et détecte au tour d ; comme d est au plus égal au diamètre, la boucle de `diametre+1` tours suffit. (3) Vérification sur la figure 2 : D et E reçoivent le *search* de A au tour 0 ; au tour 1 D reçoit de E un *search* de parent A, égal à son père, donc D a pour frère E (et réciproquement). H reçoit de K un *search* de parent D alors que son père est E : pas frère. On retrouve {D, E}.

Exercice 2 - Routage avec tables de routage (5 points)

2.2 Traitement des paquets de données reçus (5 points)

Code à placer à l'endroit « Code à remplir ici » (après les deux appels de communication et la remise à `pNull` de `messDataOut`). Un paquet est *null* si son champ *from* vaut -1. Le port de réception est l'indice *i* du canal dans `inData`.

```
for i := range inData {
    pq := messDataIn[i]
    if pq.from != -1 {                      // paquet non null
        fmt.Println("id:", id, "tour:", tour, "port:", i, "paquet:", pq)
        if pq.to == id {
            fmt.Println("id:", id, "- le message est arrive a destination")
        } else {
            // retransmission vers la destination, d'apres la table de routage
            indVois := table[pq.to].indVois
            if indVois != -1 {
                messDataOut[indVois] = pq    // from et to inchanges
            }
        }
    }
}
```

Remarques : l'énoncé impose au plus un paquet à transmettre par canal et par tour (pas de file d'attente), donc on écrit directement dans `messDataOut[indVois]`. Le champ *from* reste celui de la source d'origine. **Attention :** la page 5 du sujet (suite de 2.2) n'étant pas sur les photos, la partie « sinon » (retransmission) est la suite logique du texte visible ; à comparer avec la fin de l'énoncé.